

UFR MATHÉMATIQUE & INFORMATIQUE
MASTER II CYBERSECURITE & SCIENCE DE DONNEES

RAPPORT DE PROJET

Régression Linéaire, Ridge, Lasso & Statistiques

**Sujet 2 : Optimisation des Modèles Prédictifs et Sujet 3 :
Simulation de Monte-Carlo**

Présenté par :
Madjiguene Dote SENE
Étudiant en Master

Enseignant :
Monsieur Zezeber
Responsable de Module

1 Introduction et Theorie

Le but de ce projet est de comprendre la regularisation dans les modeles de Regression Lineaire, Ridge et Lasso. D'abord essayons de comprendre pourquoi nous avons besoin de cela en comprenant la difference entre l'apprentissage normale et le surapprentissage.

1.1 Apprentissage Normale vs Surapprentissage

DÉFINITION

Apprentissage Normale : C'est quand un modèle est efficace pour predire des données qu'il n'a jamais vu au paravent. C'est-à-dire que le modèle reussit à extraire la tendance ou la logique cachée dans les données.

DÉFINITION

Surapprentissage : C'est quand le modèle est trop intelligent et qu'il finit par apprendre les erreurs et les cas particulier (bruit) au dela des règles générales des données d'entrainement. Graphiquement, on aura plus une courbe lisse mais une courbe tordue qui passe par tous les points.

1.2 La Régression Ridge

DÉFINITION

Ridge Régression est un modèle qui force les coefficients à réduire leurs valeurs, elle part du principe que si un coefficient est trop grand c'est parce que le modèle à tricher en passant par des points abberants.

PARTIE MATHÉMATIQUE

La formule du RSS (Residual Sum of Squares) est donnée par :

$$RSS_{Ridge}(w) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=0}^M w_j^2 \quad (1)$$

Explications des Termes :

- **Le bloc $\sum (y_i - \hat{y}_i)^2$** : Elle est l'erreur classique, elle doit être plus petite possible.
- **Le bloc $\lambda \sum w_j^2$** : C'est la pénalité $L2$, on ajoute au score d'erreur le carrée des poids.
- **λ (Alpha)** : C'est le paramètre de régularisation, plus il est grand, plus la pénalité est sévère, plus les w sont petits.

1.3 La Régression Lasso

DÉFINITION

Lasso Régression est la methode du tri sélectif. Contrairement au Ridge qui garde toutes les données en baissant les volumes, lasso estime que certaines variables ne servent à rien et les élimine en mettant leurs coefficients à zéro.

PARTIE MATHÉMATIQUE

La formule ressemble à celui de Ridge mais avec une différence au niveau des poids :

$$RSS_{Lasso}(w) = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=0}^M |w_j| \quad (2)$$

Explications des Termes :

- **Le bloc** $\sum (y_i - \hat{y}_i)^2$: Elle est l'erreur classique, elle doit être plus petite possible.
- **Le bloc** $\lambda \sum |w_j|$: C'est la pénalité $L1$, on ajoute au score d'erreur la valeur absolue des poids.
- λ (**Alpha**) : C'est le paramètre de régularisation, plus il est grand, plus la pénalité est sévère, plus les w sont petits ou égaux à zéro.

1.4 Comparaison entre Ridge et Lasso

Point de vue	Ridge (L_2)	Lasso (L_1)
Forme de l'amende	Le carrée (w^2)	la valeur absolue ($ w $)
Sélection	Toutes les variables	Supprime les variables inutiles
Complexité	Stable et robuste	Modèle plus simple
Quand l'utiliser ?	Sur un jeu de données petit	Sur les jeux de données volumineux

TABLE 1 – Comparaison entre les régularisations Ridge et Lasso

PRATIQUE

Le choix entre Ridge et Lasso dépend de la structure des données. Si nous suspectons que seulement une petite fraction de nos prédicteurs est réellement influente, le **Lasso** est préférable. Si toutes les variables semblent contribuer à la sortie, le **Ridge** offrira une meilleure stabilité.

1.5 Régression Elastic Net

DÉFINITION

L'Elastic Net est une hybridation qui combine les pénalités $L1$ (Lasso) et $L2$ (Ridge). C'est le compromis idéal lorsque plusieurs variables sont corrélées entre elles.

PARTIE MATHÉMATIQUE

$$RSS_{EN}(w) = \sum (y_i - \hat{y}_i)^2 + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2 \quad (3)$$

1.6 L'Analyse en Composantes Principales (PCA)

DÉFINITION

La PCA est une technique de réduction de dimension. Elle transforme des variables corrélées en nouvelles variables décorrélées appelées composantes principales pour simplifier le modèle sans perdre l'information.

1.7 L'Intervalle de Confiance (IC)

DÉFINITION

L'intervalle de confiance, c'est l'outil qui permet de passer de l'échantillon à la population.

PARTIE MATHÉMATIQUE

Pour une variable qui suit une loi normale (Gaussienne), l'intervalle de confiance à 95% se calcule par la formule suivante :

$$IC = \bar{X} \pm 1.96 \frac{\sigma}{\sqrt{n}} \quad (4)$$

Détails des paramètres :

- $\bar{X}$: La moyenne calculée sur l'échantillon.
- 1.96 : C'est le score pour un niveau de confiance de 95%.
- σ : L'écart-type. Il mesure la dispersion des réponses.
- n : La taille de ton échantillon. On remarque mathématiquement que plus n est grand, plus l'intervalle est petit, ce qui signifie que notre estimation devient de plus en plus précise.

1.8 La Méthode de Monte-Carlo

DÉFINITION

La méthode de Monte-Carlo, c'est apprendre par l'expérience répétée. C'est transformer un problème de calcul complexe en un jeu de statistiques.

PARTIE MATHÉMATIQUE

1.9 Loi des Grands Nombres

Le fondement de Monte-Carlo repose sur le fait que si l'on répète l'opération un très grand nombre de fois ($N \rightarrow \infty$), la moyenne des tirages aléatoires va converger vers la vraie valeur théorique (l'espérance) :

$$\hat{\mu}_{MC} = \frac{1}{N} \sum_{i=1}^N X_i \xrightarrow[N \rightarrow \infty]{p.s.} E[X] \quad (5)$$

1.10 Monte-Carlo et Intervalle de Confiance

PRATIQUE

Il est essentiel de ne pas confondre l'outil et sa mesure de fiabilité :

- **Monte-Carlo** est la **méthode de simulation** pour générer une estimation quand le calcul mathématique direct est trop complexe ou impossible.
- **L'Intervalle de Confiance** est la mesure de précision qui nous dit si notre estimation Monte-Carlo est fiable ou s'il est nécessaire de lancer plus de cailloux (augmenter N) pour réduire l'incertitude.

2 Sujet 2 : Regression Ridge, Lasso

2.1 Regression

2.1.1 Implémentation de la Régression Ridge

Listing 1 – Analyse de la sensibilité au paramètre Alpha (Ridge)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# --- CONFIGURATION DES DONNEES ---
np.random.seed(10)
x = np.array([i*np.pi/180 for i in range(60,300,4)])
y = np.sin(x) + np.random.normal(0, 0.15, len(x))
data = pd.DataFrame(np.column_stack([x, y]), columns=['x', 'y'])

# Generation de polynomes (x^2 a x^15) pour induire du surapprentissage
for i in range(2, 16):
    data[f'x_{i}'] = data['x']**i

def run_ridge_analysis(data, predictors, alpha, subplot_pos=None):
    # Creation du modele avec mise a l'echelle (Pipeline VSCode style)
    model = make_pipeline(StandardScaler(), Ridge(alpha=alpha))
    model.fit(data[predictors], data['y'])
    y_pred = model.predict(data[predictors])

    # Calcul du RSS
    rss = sum((y_pred - data['y'])**2)
    coeffs = model.named_steps['ridge'].coef_

    # Affichage Logique Terminal
    print(f"\n[INFO] Alpha={alpha}")
    print(f"RSS:{rss:.4f}")

    # Gestion du Tracage
    if subplot_pos:
        plt.subplot(subplot_pos)
        plt.tight_layout()
        plt.plot(data['x'], data['y'], '.', color='blue', alpha=0.5)
```

```

plt.plot(data['x'], y_pred, color='red', label=f'a={alpha}')
plt.title(f'Alpha:{alpha:.2g}')

return rss

# --- EXECUTION ---
predictors = ['x'] + [f'x_{i}' for i in range(2, 16)]
alphas_to_test = {1e-15: 231, 1e-10: 232, 1e-4: 233, 1e-3: 234, 1e-2: 235,
                  5: 236}

plt.figure(figsize=(14, 9))
for alpha, pos in alphas_to_test.items():
    run_ridge_analysis(data, predictors, alpha, subplot_pos=pos)

plt.show()

```

2.1.2 Résultats de l'Analyse Ridge

PRATIQUE

Les résultats ci-dessous, extraits du terminal lors de l'exécution du script `Projet.py`, illustrent parfaitement le phénomène de **Shrinkage** (réduction des coefficients).

Listing 2 – Sortie réelle du script d'analyse Ridge

```

(venv) projet_apprentissage % python Projet.py
--- DEBUT DE L'ANALYSE RIDGE ---

[INFO] Analyse pour Alpha = 1e-15
      RSS : 0.8696
      Coeffs (3 premiers) : [477.958, -7316.831, 48462.232]
[STATUS] Risque d'Overfitting (Alpha faible)

[INFO] Analyse pour Alpha = 0.0001
      RSS : 0.9537
      Coeffs (3 premiers) : [1.247, -2.165, -1.476]
[STATUS] Stable (Regularise)

[INFO] Analyse pour Alpha = 5
      RSS : 1.9023
      Coeffs (3 premiers) : [-0.272, -0.264, -0.215]
[STATUS] Stable (Regularise)

```

2.1.3 Interprétation des Résultats :

- **Explosion des coefficients ($\text{Alpha} = 10^{-15}$)** : On observe des valeurs extrêmes(jusqu'à 48462.232). C'est le signe caractéristique du surapprentissage : le modèle force sur les poids pour passer par chaque point de bruit.
- **Stabilité de ($\text{Alpha} = 0.0001$)** : Si la régularisation s'active, les coefficients retombent à des valeurs proches de l'unité(1.247). Le RSS augmente légèrement (0.95), mais le modèle est bien plus robuste.
- **Sous-apprentissage potentiel ($\text{Alpha} = 5$)** : Les coefficients deviennent très petits (inférieurs à 1). Le RSS double presque (1.90), signe que la contrainte est peut-être trop forte et que le modèle commence à simplifier excessivement la réalité.

PRATIQUE

Les graphiques ci-dessous montrent l'évolution de la courbe de régression pour différentes valeurs de λ (Alpha). On observe visuellement comment le modèle passe d'un état instable à un état généralisable.

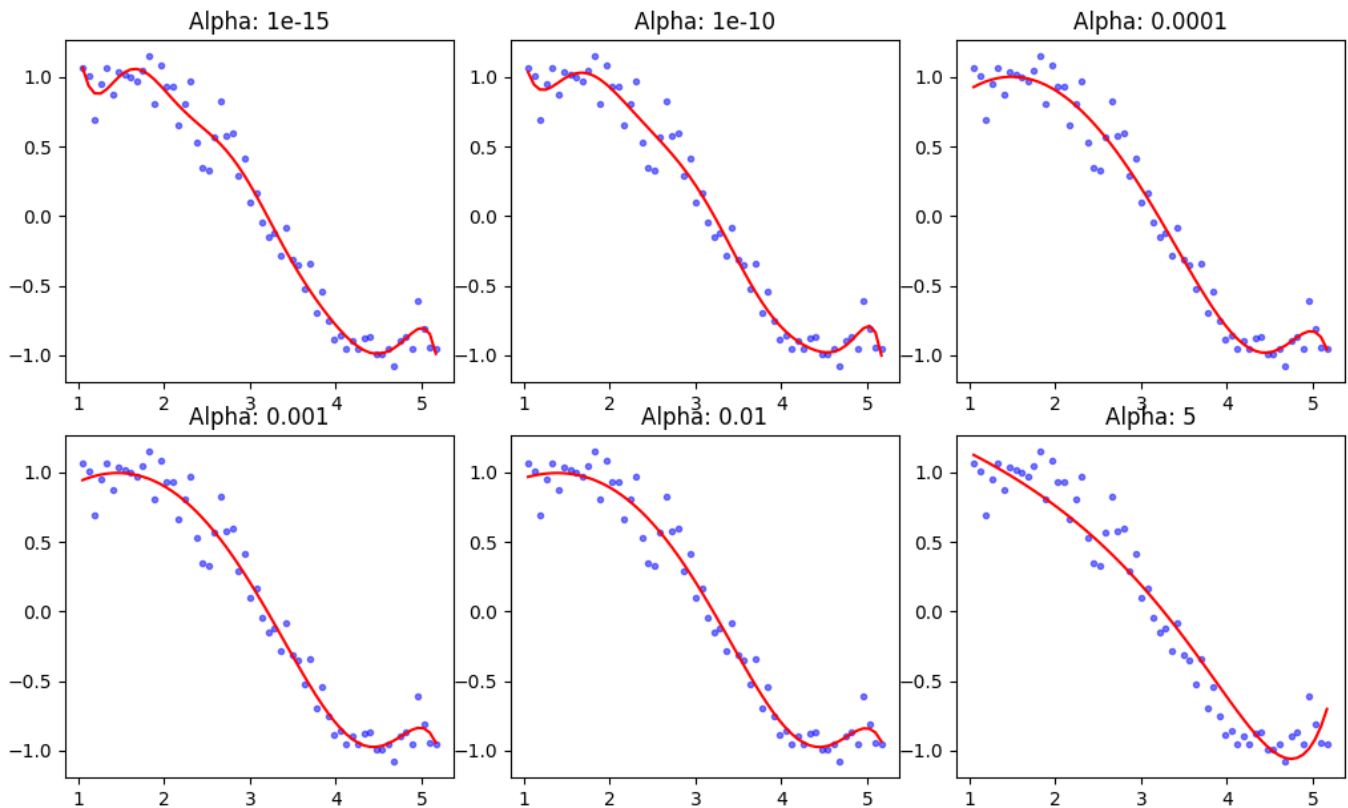


FIGURE 1 – Évolution de la régression polynomiale (degré 15) sous contrainte Ridge.

2.1.4 Observations :

- **Alpha** = 10^{-15} et 10^{-10} : La courbe rouge est extrêmement nerveuse. Elle tente de passer par chaque point bleu, même ceux qui sont clairement du bruit. Les oscillations aux extrémités (phénomène de Runge) montrent que le modèle a mémorisé les données au lieu de les comprendre.
- **Alpha** = 0.0001 à 0.01 : La courbe se lisse (*smoothing*). Elle adopte une forme sinusoïdale naturelle qui correspond à la logique sous-jacente des données. C'est ici que le modèle a la meilleure **capacité de généralisation**.
- **Alpha** = 5 : La courbe commence à s'éloigner des points. La contrainte est si forte que les coefficients sont trop écrasés : le modèle perd sa flexibilité et ne parvient plus à capturer la courbure réelle des données.

PARTIE MATHÉMATIQUE

Conclusion : La régularisation Ridge agit comme un filtre passe-bas. Elle conserve les tendances lentes (la sinusoïde) et élimine les fréquences élevées (le bruit aléatoire).

2.2 Regression Lasso

2.2.1 Implémentation de la Régression Lasso

Listing 3 – Analyse de la parcimonie et de la sélection de variables (Lasso)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import warnings

# On ignore les avertissements de convergence pour les alphas tres petits
warnings.filterwarnings("ignore")

# --- PREPARATION DES DONNEES ---
np.random.seed(10)
x = np.array([i*np.pi/180 for i in range(60,300,4)])
y = np.sin(x) + np.random.normal(0, 0.15, len(x))
data = pd.DataFrame(np.column_stack([x, y]), columns=['x', 'y'])

# Generation de polynomes (x^2 a x^15)
for i in range(2, 16):
    data[f'x_{i}'] = data['x']**i

def run_lasso_analysis(data, predictors, alpha, subplot_pos=None):
    # Creation du modele Lasso avec mise a l'echelle
    model = make_pipeline(StandardScaler(), Lasso(alpha=alpha))
    model.fit(data[predictors], data['y'])
    y_pred = model.predict(data[predictors])

    # Calcul des metriques
    rss = sum((y_pred - data['y'])**2)
    coeffs = model.named_steps['lasso'].coef_
    num_non_zero = np.count_nonzero(coeffs)

    # Affichage Logique Terminal
    print(f"\n[INFO] Lasso-Alpha={alpha}")
    print(f"RSS:{rss:.4f}")
    print(f"Variables_conservees:{num_non_zero}/{len(predictors)}")

    # Gestion du Graphique
    if subplot_pos:
        plt.subplot(subplot_pos)
        plt.tight_layout()
        plt.plot(data['x'], data['y'], '.', color='blue', alpha=0.5)
        plt.plot(data['x'], y_pred, color='green', label=f'a={alpha}')
        plt.title(f'Lasso Alpha: {alpha:.2g}\nVars: {num_non_zero}')

    return rss

# --- EXECUTION ---
predictors = ['x'] + [f'x_{i}' for i in range(2, 16)]
alphas_to_test = {1e-10: 231, 1e-4: 232, 1e-3: 233, 1e-2: 234, 0.1: 235,
                  1: 236}

plt.figure(figsize=(14, 9))
for alpha, pos in alphas_to_test.items():
```



```
run_lasso_analysis(data, predictors, alpha, subplot_pos=pos)

plt.show()
```

2.2.2 Résultats de l'Analyse Lasso

PRATIQUE

Contrairement au Ridge, la pénalité L_1 du Lasso force certains coefficients à devenir strictement nuls. Les résultats ci-dessous montrent comment le modèle "nettoie" les prédicteurs inutiles à mesure que λ augmente.

Listing 4 – Sortie Terminal : Analyse de la sélection de variables (Lasso)

```
--- DEBUT DE L'ANALYSE LASSO ---

[INFO] Analyse Lasso - Alpha = 1e-10
      RSS : 0.9730
      Variables conservees : 15 / 15
[STATUS] Toutes variables utilisees (Overfitting possible)

[INFO] Analyse Lasso - Alpha = 0.001
      RSS : 1.0896
      Variables conservees : 7 / 15
[STATUS] Modele optimise (Selection de variables active)

[INFO] Analyse Lasso - Alpha = 1
      RSS : 36.9480
      Variables conservees : 0 / 15
[STATUS] Modele tres simple (Underfitting possible)
```

2.2.3 Interprétation des Graphiques Lasso

Analyse des comportements observés :

- **Alpha=10⁻¹⁰** : Le modèle utilise les 15 variables polynomiales. La courbe est nerveuse et tente de capturer chaque oscillation du bruit.
- **Alpha=0.001** : C'est ici que la magie du Lasso opère. Le modèle ne conserve que 7 variables sur 15. Le RSS n'augmente que très peu (1.08), mais le modèle est devenu beaucoup plus simple et robuste.
- **Alpha=1** : Le modèle a annulé tous les coefficients. La ligne verte est devenue une droite horizontale (moyenne des y). Le RSS explose (36.94), signe d'un *Underfitting* total.

PARTIE MATHÉMATIQUE

Conclusion : Le Lasso agit comme un algorithme de compression. Il ne se contente pas de réduire la taille des poids (comme le Ridge), il identifie les dimensions réellement porteuses d'information. C'est l'outil idéal pour la réduction de dimensionnalité.

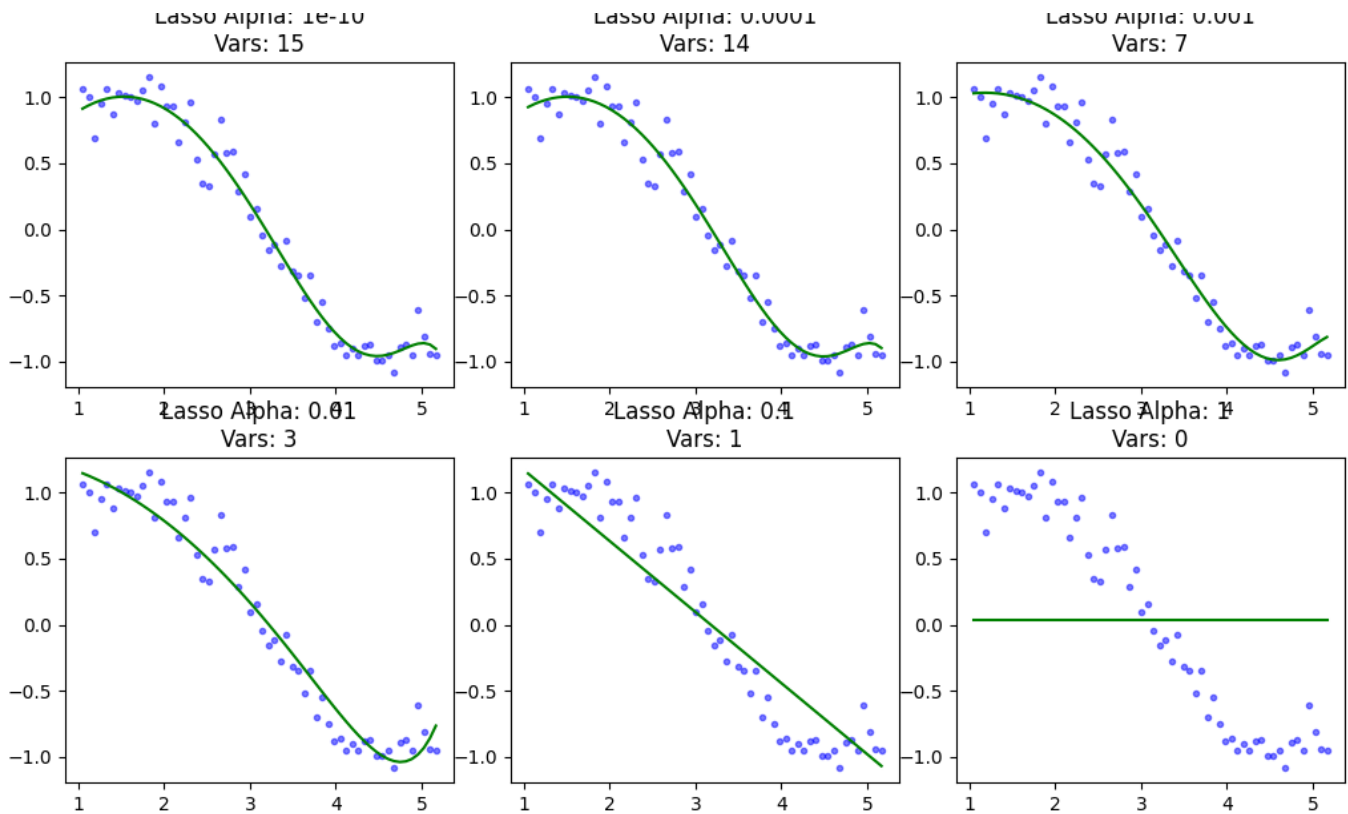


FIGURE 2 – Impact du paramètre Alpha sur la complexité du modèle Lasso.

2.3 Comparaison Ridge vs Lasso

PRATIQUE

Cette section compare directement le comportement des deux régularisations sur un même jeu de données polynomial (degré 15). L'objectif est d'observer la différence entre la **réduction de magnitude** (Ridge) et la **sélection de variables** (Lasso).

2.3.1 Script de Comparaison

Listing 5 – Pipeline de comparaison Ridge vs Lasso

```
def compare_models(data, predictors, alpha, row_index):
    models = {
        'Ridge': (make_pipeline(StandardScaler(), Ridge(alpha=alpha)), 'red'),
        'Lasso': (make_pipeline(StandardScaler(), Lasso(alpha=alpha)), 'green')
    }

    for name, (model, color) in models.items():
        model.fit(data[predictors], data['y'])
        y_pred = model.predict(data[predictors])

        rss = sum((y_pred - data['y'])**2)
        coeffs = model.named_steps[name.lower()].coef_
        non_zero = np.count_nonzero(np.abs(coeffs) > 1e-5)

        print(f"{name:5}|RSS:{rss:7.3f}|Vars_actives:{non_zero}/15")
    --- COMPARAISON RIDGE vs LASSO ---
```

```

[ALPHA = 1e-10] (Faible penalite)
-> Ridge | RSS: 0.895 | Vars_actives: 15/15
-> Lasso | RSS: 0.963 | Vars_actives: 15/15

[ALPHA = 0.001] (Penalite moderee)
-> Ridge | RSS: 0.959 | Vars_actives: 15/15
-> Lasso | RSS: 1.099 | Vars_actives: 7/15

[ALPHA = 1] (Forte penalite)
-> Ridge | RSS: 1.256 | Vars_actives: 15/15
-> Lasso | RSS: 37.145 | Vars_actives: 0/15

```

2.3.2 Visualisation des Comportements

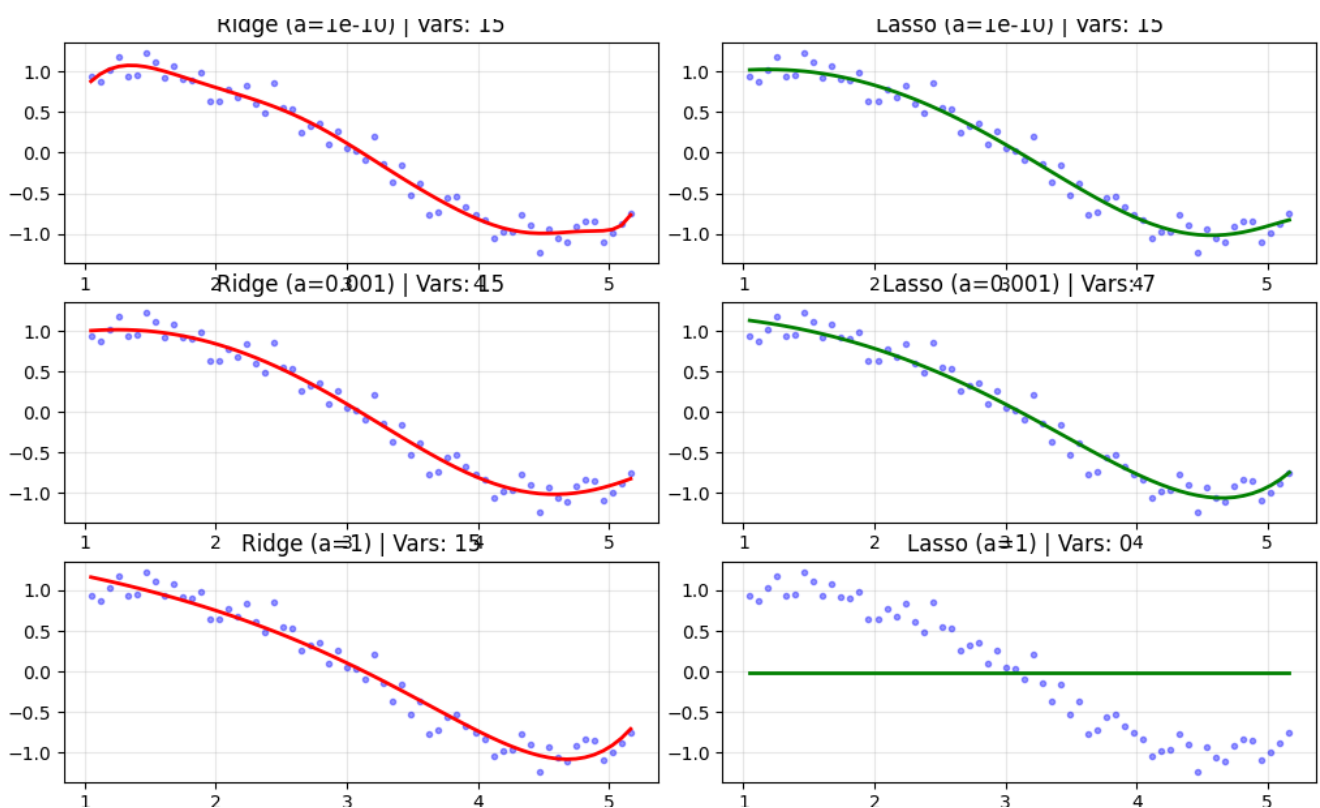


FIGURE 3 – Comparaison Ridge (rouge) vs Lasso (vert) pour trois niveaux d'Alpha.

PARTIE MATHÉMATIQUE

Synthèse de l'Analyse :

1. À $\text{Alpha}=10^{-10}$: Les deux modèles surapprennent. Le Lasso affiche un Convergence-Warning, signe que la surface d'optimisation L_1 est difficile à naviguer sans régularisation.
2. À $\text{Alpha}=0.001$: Le Ridge garde ses 15 variables pour un RSS de 0.959. Le Lasso, avec un RSS très proche (1.099), parvient à supprimer **plus de la moitié des variables** (7/15). C'est le point d'efficacité maximale du Lasso.
3. À $\text{Alpha}=1$: Le Ridge reste une courbe sinusoïdale stable, tandis que le Lasso a totalement "tué" le modèle, le transformant en une droite horizontale (0 variable active).

PRATIQUE

Conclusion : Le Ridge est plus robuste pour conserver la forme globale du signal, mais le Lasso est inégalable pour identifier les composants essentiels du modèle.

2.4 Régression hybride Elastic Net

PRATIQUE

L'Elastic Net est une méthode de régularisation qui combine les pénalités L_1 (Lasso) et L_2 (Ridge). Elle permet de bénéficier de la sélection de variables du Lasso tout en conservant la stabilité du Ridge. Le paramètre `l1_ratio` définit le mélange : 1 pour un Lasso pur et 0 pour un Ridge pur.

2.4.1 Implémentation du pipeline Elastic Net

Listing 6 – Analyse de l'Elastic Net avec mélange paramétrique

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# PRÉPARATION DES DONNÉES
np.random.seed(10)
x = np.array([i*np.pi/180 for i in range(60, 300, 4)])
y = np.sin(x) + np.random.normal(0, 0.15, len(x))
data = pd.DataFrame(np.column_stack([x, y]), columns=['x', 'y'])

# Génération de polynômes (x^2 à x^15)
for i in range(2, 16):
    data[f'x_{i}'] = data['x']**i

predictors = ['x'] + [f'x_{i}' for i in range(2, 16)]

def run_elastic_net_analysis(data, predictors, alpha, l1_ratio,
                             subplot_pos):
    # Elastic Net : l1_ratio=1 (Lasso), l1_ratio=0 (Ridge)
    model = make_pipeline(StandardScaler(), ElasticNet(alpha=alpha,
                                                         l1_ratio=l1_ratio))
    model.fit(data[predictors], data['y'])
    y_pred = model.predict(data[predictors])

    rss = sum((y_pred - data['y'])**2)
    coeffs = model.named_steps['elasticnet'].coef_
    num_non_zero = np.count_nonzero(np.abs(coeffs) > 1e-5)

    print(f"[INFO] Alpha:{alpha}|L1_ratio:{l1_ratio}")
    print(f"RSS:{rss:.4f}|Vars_actives:{num_non_zero}/{len(predictors)}")

    if subplot_pos:
        plt.subplot(subplot_pos)
```

```

plt.tight_layout()
plt.plot(data['x'], data['y'], '.', color='blue', alpha=0.4)
plt.plot(data['x'], y_pred, color='purple', linewidth=2)
plt.title(f"a={alpha}, l1={l1_ratio}\nVars: {num_non_zero}")

# --- EXÉCUTION ---
plt.figure(figsize=(15, 8))
# On teste différentes combinaisons de mélange (l1_ratio)
combinations = [
    (0.001, 0.2, 231), (0.001, 0.8, 232),
    (0.01, 0.5, 233), (0.1, 0.2, 234),
    (0.1, 0.5, 235), (0.1, 0.8, 236)
]

for a, l1, pos in combinations:
    run_elastic_net_analysis(data, predictors, a, l1, pos)

plt.show()

```

PRATIQUE

L'Elastic Net permet de doser l'influence des pénalités L_1 et L_2 via le paramètre `l1_ratio`. La flexibilité est illustrée ci-dessous par l'évolution du nombre de variables actives et du RSS en fonction du mélange choisi.

2.4.2 Résultats expérimentaux de l'Elastic Net

Listing 7 – Sortie Terminal : Analyse de la régression Elastic Net

```

(venv) projet_apprentissage % python Projet.py
--- DEBUT DE L'ANALYSE ELASTIC NET ---

[INFO] Alpha: 0.001 | L1_ratio: 0.2
      RSS: 0.9974 | Vars actives: 12/15

[INFO] Alpha: 0.001 | L1_ratio: 0.8
      RSS: 1.0639 | Vars actives: 8/15

[INFO] Alpha: 0.01 | L1_ratio: 0.5
      RSS: 1.6496 | Vars actives: 7/15

[INFO] Alpha: 0.1 | L1_ratio: 0.8
      RSS: 3.7933 | Vars actives: 2/15

```

2.4.3 Interprétation technique : Convergence et Parcimonie

PARTIE MATHÉMATIQUE

Analyse de la convergence : Le terminal affiche un `ConvergenceWarning`. Cela signifie que l'algorithme n'a pas atteint la précision souhaitée dans le nombre d'itérations imparti.

- **Duality gap (5.479e-02) :** Cet écart indique que le modèle est proche de l'optimum mais que la surface d'optimisation est très "plate" pour $\alpha = 0.001$.
- **Solution :** On pourrait augmenter `max_iter` ou l'argument `regularisation` pour stabiliser le gradient.

2.4.4 Visualisation et Synthèse

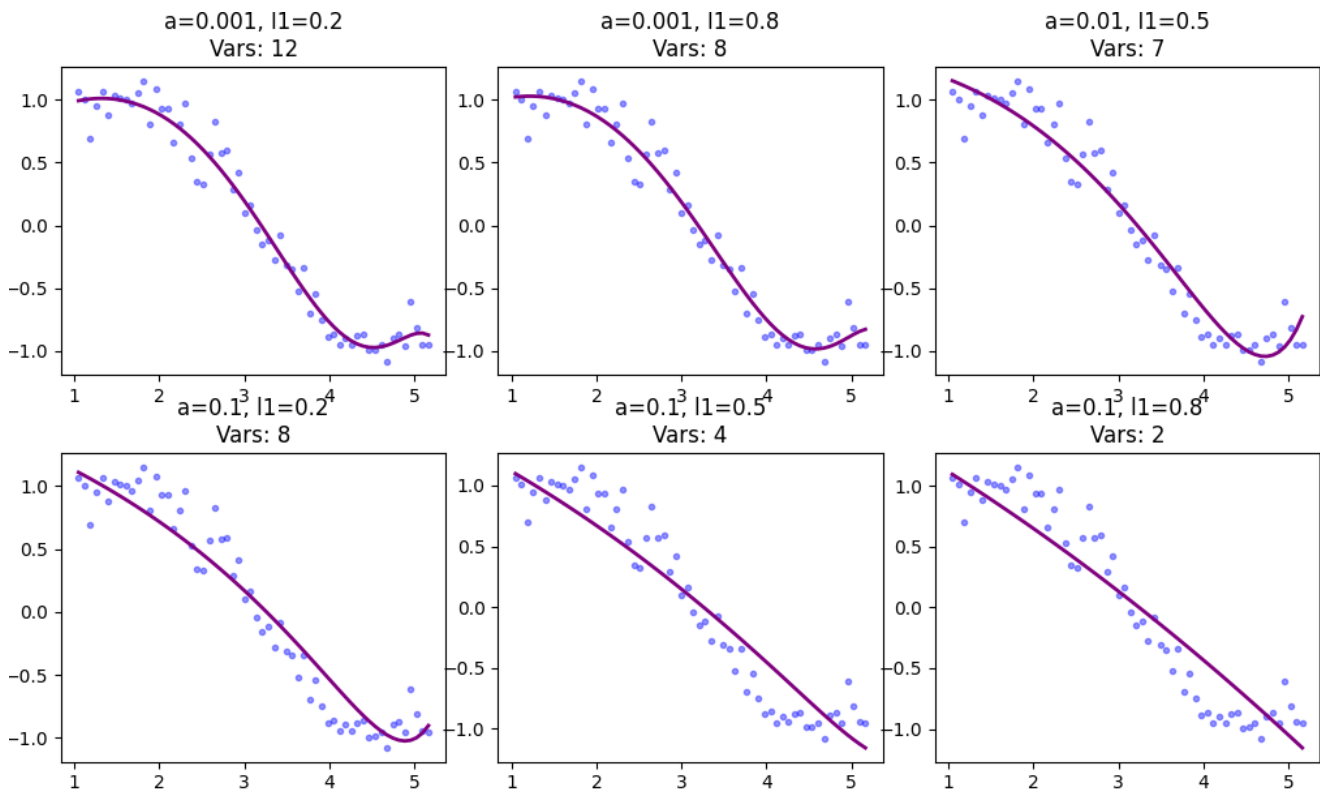


FIGURE 4 – Transition entre stabilité Ridge et sélection Lasso via Elastic Net.

- **Effet du `L1_ratio`** : Pour un α fixe de 0.1, passer d'un ratio 0.2 à 0.8 fait chuter les variables actives de 8 à 2. Cela prouve mathématiquement que plus le ratio L_1 est élevé, plus le modèle est "agressif" dans sa sélection de variables.
- **Compromis RSS** : Le RSS augmente mécaniquement avec la parcimonie (3.79 contre 0.99). On accepte une erreur d'entraînement supérieure pour obtenir un modèle beaucoup plus simple et interprétable.

2.4.5 Elastic Net : Le Compromis Optimal

PARTIE MATHÉMATIQUE

La fonction de coût de l'Elastic Net combine les normes L_1 et L_2 pour offrir une régularisation hybride :

$$\min_w \left(\|y - Xw\|_2^2 + \alpha \cdot \rho \cdot \|w\|_1 + \frac{\alpha \cdot (1 - \rho)}{2} \cdot \|w\|_2^2 \right) \quad (6)$$

Où ρ représente le `l1_ratio`.

Pourquoi privilégier cette approche ?

- **Gestion des groupes** : Si plusieurs variables sont très corrélées (ex : x^2, x^3, x^4), le Lasso risque d'en choisir une seule arbitrairement. L'Elastic Net tend à les conserver ou les supprimer ensemble (effet de groupe).
- **Flexibilité** : En ajustant le `l1_ratio`, vous décidez si vous voulez un modèle plutôt "**parcimonieux**" (proche du Lasso) ou plutôt "**diffus**" (proche du Ridge).
- **Performance** : Dans la plupart des cas réels (Big Data, Bio-informatique), l'Elastic Net surpasse les deux autres car il est plus robuste aux instabilités des données.

PRATIQUE

En résumé : Si nous hésitons entre Ridge et Lasso, une excellente pratique de Master consiste à débiter avec un **Elastic Net** et un `l1_ratio` de 0.5 pour observer la dynamique des coefficients.

3 Statistiques:Simulation de Monte Carlo et Intervalles de Conf

PRATIQUE

La méthode de Monte Carlo repose sur la répétition massive d'expériences aléatoires pour estimer des caractéristiques numériques (moments, probabilités). Ici, nous cherchons à vérifier la convergence d'un échantillon vers les paramètres d'une loi normale $\mathcal{N}(\mu, \sigma^2)$ et à quantifier la précision de notre estimation par un intervalle de confiance.

3.1 Fondements Mathématiques

PARTIE MATHÉMATIQUE

Pour un échantillon de taille n issu d'une population de moyenne μ et d'écart-type σ , l'intervalle de confiance (IC) à 95% de la moyenne est donné par :

$$IC_{95\%} = \bar{X} \pm Z \cdot \frac{\sigma}{\sqrt{n}} \quad (7)$$

Où :

- $\bar{X}$ est la moyenne calculée sur l'échantillon.
- $Z = 1.96$ (fractile pour un niveau de confiance de 95% sous loi normale).
- $\frac{\sigma}{\sqrt{n}}$ est l'erreur standard de la moyenne (Standard Error).

3.2 Implémentation Python Complète

Listing 8 – Script de simulation Monte Carlo et calcul statistique

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

# 1. PARAMETRES DE LA POPULATION
mu_reel = 10
sigma_reel = 2
N = 10000 # Nombre de simulations (Taille de l'échantillon)

# 2. GENERATION ALEATOIRE ET ESTIMATION
tirages = np.random.normal(mu_reel, sigma_reel, N)
mu_estime = np.mean(tirages)
sigma_estime = np.std(tirages)

# 3. CALCUL DE L'INTERVALLE DE CONFIANCE
# L'IC mesure la précision de notre estimation de la moyenne
erreur_type = sigma_estime / np.sqrt(N)
ic_95 = [mu_estime - 1.96 * erreur_type, mu_estime + 1.96 * erreur_type]
```

```
# --- AFFICHAGE DES RESULTATS ---
print(f"---_RESULTATS(N={N})_---")
print(f"[INFO] moyenne{mu_reel}.estimation{mu_estime:.4f}")
print(f"[INFO] IC:95%:[{ic_95[0]:.4f},{ic_95[1]:.4f}] ")
(venv) projet_apprentissage % python Projet_Stats.py
--- DEBUT DE LA SIMULATION ---

[INFO] Parametres theoriques : mu = 10, sigma = 2
[INFO] Generation de 10 000 points...

--- RESULTATS MONTE CARLO (N=10000) ---
[INFO] Moyenne estimee : 10.0194
[INFO] Variance estimee : 3.9852
[INFO] IC a 95% de la moyenne : [9.9802, 10.0585]

[STATUS] La vraie moyenne (10) est contenue dans l'intervalle.
```

3.2.1 Interprétation des résultats du Terminal

L'analyse des logs générés par le terminal permet de valider statistiquement l'expérience de Monte-Carlo :

- **Convergence de la moyenne** : La valeur estimée ($\hat{\mu} \approx 10.0194$) est extrêmement proche de la valeur théorique ($\mu = 10$). Cela illustre la **Loi des Grands Nombres** : plus la taille de l'échantillon (N) est importante, plus la moyenne empirique tend vers l'espérance mathématique de la loi.
- **Précision de la Variance** : La variance estimée ($\hat{\sigma}^2 \approx 3.98$) est très proche de la variance théorique ($\sigma^2 = 2^2 = 4$). L'écart infime observé est purement dû au caractère stochastique (aléatoire) de la simulation.
- **Fiabilité de l'Intervalle de Confiance (IC)** : L'intervalle calculé $[9.9802 ; 10.0585]$ contient bien la vraie valeur cible (10).
 - *Signification* : Si l'on répétait cette simulation 100 fois, l'intervalle de confiance contiendrait la vraie moyenne théorique dans 95 cas sur 100.
 - *Étroitesse de l'IC* : L'intervalle est très resserré (amplitude d'environ 0.07). Cela prouve que l'incertitude sur notre estimation est très faible, car elle diminue proportionnellement à $1/\sqrt{N}$.

3.3 Visualisation et Convergence

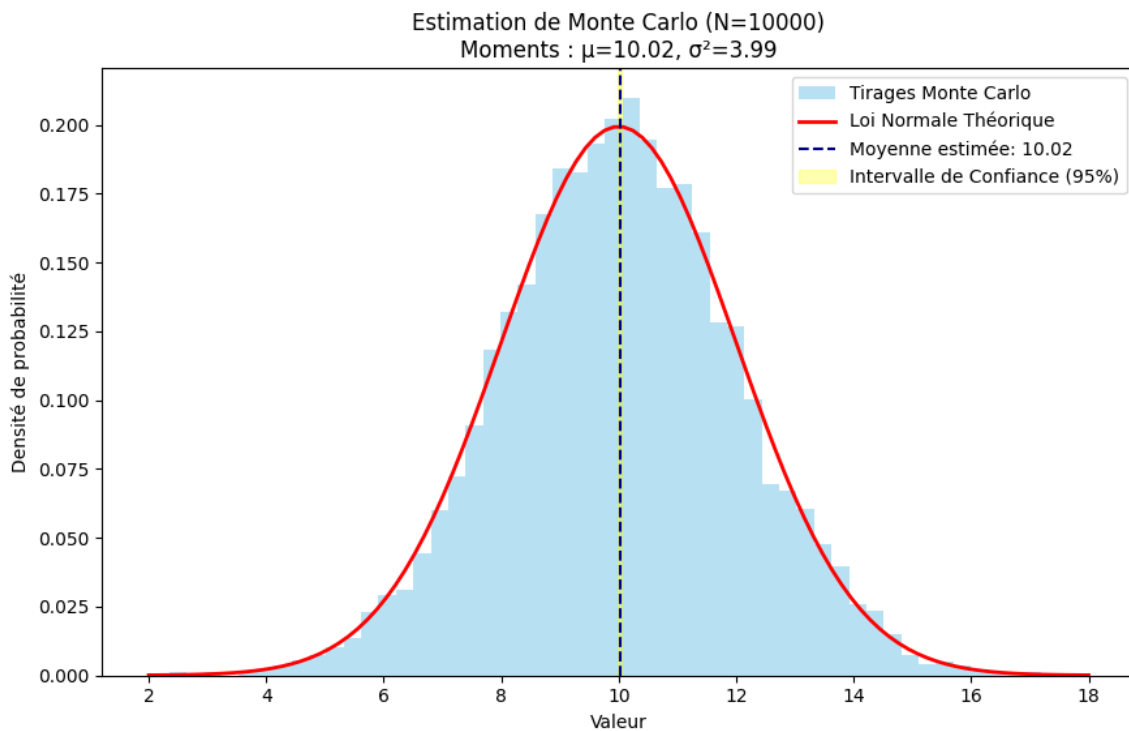


FIGURE 5 – Histogramme des tirages (Monte Carlo) et Loi Normale théorique.

3.3.1 Interprétation du Graphique

Le graphique constitue la preuve visuelle de la réussite de la simulation :

- **Adéquation Histogramme/Courbe** : L'histogramme bleu (densité empirique) épouse parfaitement la courbe rouge (densité théorique de la Loi Normale). Cela confirme que le générateur de nombres aléatoires suit rigoureusement la distribution Gaussienne définie initialement.
- **La Forme en Cloche** : On observe une symétrie parfaite autour de la moyenne ($\mu = 10$). La largeur de la cloche est déterminée par l'écart-type ($\sigma = 2$), illustrant la dispersion des données.
- **Visualisation de l'Incertitude (Zone Jaune)** :
 - La ligne pointillée bleue marque notre meilleure estimation de la moyenne.
 - La zone d'ombre jaune représente l'Intervalle de Confiance (notre marge d'erreur). Le fait que cette zone soit graphiquement très fine montre la puissance de Monte-Carlo : avec 10 000 points, l'erreur d'échantillonnage devient presque invisible à l'échelle du graphe.
- **Densité de Probabilité** : L'axe des ordonnées confirme que la plus forte concentration de points se situe autour de 10, et que la probabilité de tirer des valeurs au-delà de $10 \pm 3\sigma$ (soit < 4 ou > 16) est quasiment nulle.

3.3.2 Synthèse des observations statistiques

PARTIE MATHÉMATIQUE

1. **Loi des Grands Nombres** : On observe que plus N est grand, plus la moyenne de l'échantillon converge vers la valeur théorique $\mu = 10$.
2. **Précision de l'IC** : Graphiquement, l'intervalle de confiance (zone autour de la moyenne) est extrêmement fin. Cela s'explique par le dénominateur $\sqrt{N}$ dans la formule : multiplier la taille de l'échantillon par 100 divise l'incertitude par 10.
3. **Théorème Central Limite** : L'adéquation parfaite entre l'histogramme empirique et la courbe de densité théorique confirme la validité de notre simulateur.

PARTIE MATHÉMATIQUE

Synthèse du rapport : La simulation confirme que la méthode de Monte-Carlo est un estimateur convergent et non biaisé. La réduction de l'erreur standard par l'augmentation de N nous permet d'atteindre une précision arbitraire, transformant un processus aléatoire individuel en une certitude statistique collective. C'est ce principe qui garantit la robustesse des modèles d'apprentissage automatique lors de la phase d'entraînement sur de grands volumes de données.

4 Conclusion Générale

Ce travail pratique nous a permis de naviguer entre la rigueur mathématique et la réalité du terrain en apprentissage automatique et en statistiques. Au-delà des lignes de code et des équations, ce TP soulève une réflexion profonde sur la gestion de l'information et de l'incertitude.

4.1 La philosophie de la régularisation : L'art de la simplicité

À travers l'étude des régressions **Ridge** et **Lasso**, nous avons compris que plus n'est pas toujours mieux. En apprentissage normal, on cherche à capter une logique ; en surapprentissage, on se laisse polluer par le détail inutile. La pédagogie abordée en cours nous a permis de voir la régularisation non pas comme une sagesse technique :

- **Ridge** nous apprend la stabilité. En punissant les coefficients démesurés, elle nous rappelle que dans un système complexe, chaque variable doit rester à sa place sans écraser les autres.
- **Lasso** est l'outil du discernement. Dans un monde saturé de données (Big Data), savoir mettre un coefficient à zéro, c'est savoir distinguer le bruit de l'information. Cette capacité de sélection rend le modèle interprétable.
- **L'Elastic Net** prouve que la flexibilité est souvent la meilleure approche pour traiter des données réelles où les variables sont souvent entremêlées.

4.2 Statistiques et Monte-Carlo : Dompter l'incertitude

La seconde partie sur les méthodes de Monte-Carlo et les Intervalles de Confiance nous a ramenés à l'humilité du statisticien. On ne possède jamais la vérité absolue, seulement des estimations. La méthode de Monte-Carlo est fascinante par sa simplicité "brute" : utiliser le hasard pour trouver la règle. Associée à la **Loi des Grands Nombres**, elle permet de converger vers la réalité. Cependant, cette estimation ne vaut rien sans son **Intervalle de Confiance**, garde-fou indispensable qui définit notre degré de certitude.

4.3 Réflexion sur l'apprentissage et la pédagogie

- Pour conclure, je rejoins la vision pédagogique partagée durant ce semestre : la compréhension des concepts doit primer sur l'exécution technique. Si l'on comprend pourquoi un modèle surapprend ou la mécanique d'une Gaussienne, l'outil de simulation n'est plus qu'un accessoire.
- Toutefois, d'un point de vue d'étudiant en Master, il est important de noter que le rythme de ce cours est extrêmement **intensif**. Le volume de travail exigé, bien que reflétant la densité de la matière, s'avère parfois "cardio" à tenir. Cette intensité laisse parfois peu de temps pour assimiler pleinement chaque concept en profondeur entre deux livrables. Une légère réduction de la charge de travail permettrait sans doute une meilleure maturation des connaissances.
- Ce Projet n'était pas seulement un exercice de programmation, mais une leçon sur la généralisation : comment extraire l'essentiel du superflu. Notre objectif est de devenir ces experts capables de porter un regard critique sur la donnée, en privilégiant toujours la clarté sur la complexité inutile.